

# Computational Aspects of Games

Luca Dall'Asta

DISAT  
Politecnico di Torino, Italy

November 2025

# Solution techniques for games

Computational approaches depend on the game and solution concept:

- Strategic domination  $\implies$  **linear program**
- NE of zero-sum two-player games  $\implies$  linear program
- NE of general-sum two-player games  
 $\implies$  **linear complementarity problem** / non-convex quadratic program
- NE of general-sum ( $n > 2$ )-player games  
 $\implies$  non-linear complementarity problem / global optimization methods
- SPE in extensive-form games  $\implies$  **dynamic programming** (backward induction)
- NE in extensive-form games  $\implies$  LP and LCP

# Linear Programming: definition

Constrained optimization problems with linear objective functions and linear constraints.

## Definition

Given a set  $\{x_1, \dots, x_n\}$  of non-negative real variables, a **linear program** is an optimization problem that takes the *canonical form*,

$$\begin{aligned} & \underset{x}{\text{maximize}} \quad \sum_{i=1}^n c_i x_i \\ & \text{subject to} \quad \begin{cases} \sum_{i=1}^n a_{ij} x_i \leq b_j & \forall j = 1, \dots, m \\ x_i \geq 0 & \forall i = 1, \dots, n. \end{cases} \end{aligned} \quad (1)$$

## Remark

It is a *feasibility problem* if there is no objective function.

# Linear Programming: definition

## Example

Consider the LP,

$$\begin{array}{ll}\text{maximize}_{x_1, x_2} & x_1 + x_2 \\ \text{subject to} & \begin{cases} 2x_1 + 3x_2 \leq 12 \\ x_1 + 3x_2 \leq 9 \\ 2x_1 + x_2 \leq 10 \\ x_i \geq 0, \quad i = 1, 2 \end{cases}\end{array}$$

corresponding to the canonical form with

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad c = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad A = \begin{pmatrix} 2 & 3 \\ 1 & 3 \\ 2 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 12 \\ 9 \\ 10 \end{pmatrix}$$

# Linear Programming: duality

## Definition

Consider the *primal* LP  $\max_x c^\top x$  subject to  $Ax \leq b$  and  $x \geq 0$ .  
The symmetric **dual LP** is

$$\begin{aligned} & \underset{y}{\text{minimize}} && b^\top y \\ & \text{subject to} && \begin{cases} A^\top y \geq c \\ y \geq 0. \end{cases} \end{aligned} \tag{2}$$

## Proposition

*If  $x$  is feasible for the primal LP and  $y$  is feasible for the dual LP, it holds that  $c^\top x \leq b^\top y$  (weak duality).*

*If one of the two LPs admits a finite optimal solution, then the other does and  $\max_x c^\top x = \min_y b^\top y$  (strong duality).*

# Linear Programming: duality

To prove duality we use:

## Corollary (of Farkas' Lemma)

Let  $A \in \mathbb{R}^{n \times m}$ ,  $b \in \mathbb{R}^n$ , then exactly one of the statements is true:

- ①  $\exists x \in \mathbb{R}^n$  such that  $Ax \geq b$  and  $x \geq 0$ ;
- ②  $\exists y \in \mathbb{R}^m$  such that  $A^\top y \geq 0$ ,  $b^\top y < 0$  and  $y \leq 0$ .

Assume that dual LP has the optimal solution  $y^*$  and suppose by absurd that duality is not true. Then  $\nexists x$  such that  $c^\top x = b^\top y^*$ . Setting  $\gamma = b^\top y^*$ , it means  $\nexists x \in \mathbb{R}^n$ ,  $x \geq 0$  such that

$$\begin{pmatrix} -A \\ c^\top \end{pmatrix} x \geq \begin{pmatrix} -b \\ \gamma \end{pmatrix}.$$

# Linear Programming: duality

Then,  $\exists y \in \mathbb{R}^m$ ,  $y \leq 0$  and  $\exists w \in \mathbb{R}$ ,  $w \leq 0$  such that

$$\begin{pmatrix} -A^\top & c \end{pmatrix} \begin{pmatrix} y \\ w \end{pmatrix} \geq 0, \quad \text{and} \quad \begin{pmatrix} -b^\top & \gamma \end{pmatrix} \begin{pmatrix} y \\ w \end{pmatrix} < 0$$

Two cases:

If  $w = 0$ , then  $A^\top (y^* - y) \geq A^\top y^* \geq c$  and  $b^\top (y^* - y) < b^\top y^*$ , implying that  $y^* - y \geq y^*$  is a better feasible solution.

If  $w < 0$ , then dividing both sides by  $w$  gives  $A^\top (y/w) \geq c$  and  $b^\top (y/w) < \gamma$ . So  $y/w$  is a feasible solution better than  $y^*$ .

It follows that duality must be valid.

## Example

The previous canonical-form LP can be written in the following dual form

$$\begin{array}{ll}\text{minimize} & 12y_1 + 9y_2 + 10y_3 \\ & y_1, y_2, y_3 \\ \text{subject to} & \begin{cases} 2y_1 + y_2 + 2y_3 \geq 1 \\ 3y_1 + 3y_2 + y_3 \geq 1 \\ y_i \geq 0, \quad i = 1, 2, 3 \end{cases}\end{array}$$



# Linear Programming: slack variables

Inequalities can be made tight introducing **slack variables**.

A canonical-form LP can be expressed in the following *standard form*:

$$\begin{aligned} & \underset{x'}{\text{maximize}} \quad \sum_{i=1}^{n+m} c'_i x'_i \\ & \text{subject to} \quad \begin{cases} \sum_{i=1}^{n+m} a'_{ij} x'_i = b'_j & \forall j = 1, \dots, m \\ x'_i \geq 0 & \forall i = 1, \dots, n+m. \end{cases} \end{aligned} \quad (3)$$

where  $c'_i = c_i$ ,  $\forall i = 1, \dots, n$ , and for each inequality  $\sum_{i=1}^n a_{ij} x_i \leq b_j$  we introduced a slack variable  $s_j \geq 0$ , such that  $\sum_{i=1}^n a_{ij} x_i + s_j = b_j$ .

Therefore  $c' = (c, 0)^\top$ ,  $x' = (x, s)^\top$ ,  $A' = (A, I)$ , and  $b' = b$ .

## Example

Introducing slack variables  $s_1, s_2, s_3$ , the previous canonical-form LP can be written in standard (augmented) form as follows

$$\begin{array}{ll} \underset{x_1, x_2}{\text{maximize}} & x_1 + x_2 \\ \text{subject to} & \begin{cases} 2x_1 + 3x_2 + s_1 = 12 \\ x_1 + 3x_2 + s_2 = 9 \\ 2x_1 + x_2 + s_3 = 10 \\ x_i \geq 0, & i = 1, 2 \\ s_i \geq 0, & i = 1, 2, 3 \end{cases} \end{array}$$

# Linear Complementarity Problems

The LCP seeks a point  $x \in \mathbb{R}^n$  satisfying a mutual exclusivity between variables and constraints.

## Definition

Given  $M \in \mathbb{R}^{n \times n}$  and  $q \in \mathbb{R}^n$ , a **linear complementarity problem** requires to find  $z \in \mathbb{R}^n$  (and  $w \in \mathbb{R}^n$ ) such that

$$\begin{cases} w_i = \sum_{j=1}^n m_{ij} z_j + q_i & \forall i = 1, \dots, n \\ \sum_{i=1}^n z_i w_i = 0 \\ z_i \geq 0, w_i \geq 0 & \forall i = 1, \dots, n. \end{cases} \quad (4)$$

## Remark

It is a quadratic problem because the complementarity condition means

$$z^\top (Mz + q) = 0.$$

# Linear Complementarity Problems

Every LP can be expressed as a LCP

$$\begin{array}{ll} \max_x c^\top x & \\ \text{s.t. } \begin{cases} Ax \leq b \\ x \geq 0, \end{cases} & \implies \begin{cases} w = Mz + q \\ z^\top w = 0 \\ w \geq 0, z \geq 0 \end{cases} \end{array}$$

by defining  $z^\top = (x, -y)$  and

$$M = \begin{pmatrix} 0 & -A^\top \\ -A & 0 \end{pmatrix}, \quad q = \begin{pmatrix} -c \\ b \end{pmatrix}.$$

## Remark

Not every LCP correspond to LP.

# Linear Complementarity Problems

From the canonical-form LP

$$\begin{array}{ll}\max_x & c^\top x \\ \text{s.t.} & \begin{cases} Ax \leq b \\ x \geq 0, \end{cases}\end{array}$$

consider the primal equality (introducing slack variables  $s \geq 0$ )

$$Ax + s = b$$

and the dual LP

$$\begin{array}{ll}\min_y & b^\top y \\ \text{s.t.} & \begin{cases} A^\top y \geq c \\ y \leq 0, \end{cases}\end{array}$$

with dual equality (introducing slack variables  $r \geq 0$ )

$$A^\top y - r = c.$$

# Linear Complementarity Problems

Define

$$\begin{aligned}\begin{pmatrix} r \\ s \end{pmatrix} &= \begin{pmatrix} A^\top y - c \\ -Ax + b \end{pmatrix} \\ &= \begin{pmatrix} 0 & -A^\top \\ -A & 0 \end{pmatrix} \begin{pmatrix} x \\ -y \end{pmatrix} + \begin{pmatrix} -c \\ b \end{pmatrix}\end{aligned}$$

We found the form  $w = Mz + q$ .

Interestingly, the complementarity condition imposes

$$\begin{cases} x_i r_i = 0 \\ x_i \geq 0, r_i \geq 0 & \forall i = 1, \dots, n \\ y_j s_j = 0 \\ -y_j \geq 0, s_j \geq 0 & \forall j = 1, \dots, m \end{cases}$$

# Linear Complementarity Problems

## Example

Consider the LP with

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad c = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad A = \begin{pmatrix} 2 & 3 \\ 1 & 3 \\ 2 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 12 \\ 9 \\ 10 \end{pmatrix}$$

the LCP is

$$\begin{cases} w_1 = -2z_3 - z_4 - 2z_5 - 1 \\ w_2 = -3z_3 - 3z_4 - z_5 - 1 \\ w_3 = -2z_1 - 3z_2 + 12 \\ w_4 = -z_1 - 3z_2 + 9 \\ w_5 = -2z_1 - z_2 + 10 \\ w_i \geq 0, z_i \geq 0 \\ \sum_i w_i z_i = 0 \end{cases} \quad i = 1, \dots, 5$$

# Computing NE of two-player zero-sum games

The (Nash) equilibrium strategy for player 1 is the maxmin strategy defined by

$$\max_{s_1} \min_{s_2} u_1(s_1, s_2) \quad (5)$$

which corresponds to solving the **Linear Program**

$$\begin{aligned} & \underset{\sigma_1}{\text{maximize}} \quad u_1^*(\sigma_1) \\ & \text{subject to} \quad \begin{cases} \sum_{a_1 \in A_1} u_1(a_1, a_2) \sigma_1(a_1) \geq u_1^* & \forall a_2 \in A_2 \\ \sum_{a_1 \in A_1} \sigma_1(a_1) = 1 \\ \sigma_1(a_1) \geq 0 & \forall a_1 \in A_1. \end{cases} \end{aligned} \quad (6)$$

Solution gives the optimal mixed strategy for player 1.



# Computing NE of two-player zero-sum games

## Example

Consider the game with payoff matrix

|     | $l$     | $r$     |
|-----|---------|---------|
| $U$ | $2, -2$ | $-1, 1$ |
| $D$ | $1, -1$ | $2, -2$ |

Given the mixed strategy  $\sigma_1 = (p, 1 - p)$ , the LP conditions are

$$\begin{cases} 2p + 1(1 - p) \geq u_1^* & \implies p + 1 \geq u_1^* \\ -1p + 2(1 - p) \geq u_1^* & \implies 2 - 3p \geq u_1^* \end{cases} \quad (7)$$

Then  $\max_p u_1^*(p)$  subject to such conditions can be found imposing marginality, i.e.  $p + 1 = 2 - 3p$ , which gives  $p = \frac{1}{4}$ .

# Computing NE of two-player zero-sum games

But it also holds that

$$\max_{s_1} \min_{s_2} u_1(s_1, s_2) = \min_{s_2} \max_{s_1} u_1(s_1, s_2) \quad (8)$$

and the r.h.s. corresponds to solving the dual LP

$$\begin{aligned} & \underset{\sigma_2}{\text{minimize}} \quad u_1^*(\sigma_2) \\ & \text{subject to} \quad \begin{cases} \sum_{a_2 \in A_2} u_1(a_1, a_2) \sigma_2(a_2) \leq u_1^* & \forall a_1 \in A_1 \\ \sum_{a_2 \in A_2} \sigma_2(a_2) = 1 \\ \sigma_2(a_2) \geq 0 & \forall a_2 \in A_2. \end{cases} \end{aligned} \quad (9)$$

Solution gives the optimal mixed strategy for player 2.

# Computing NE of two-player zero-sum games

## Example (cont'd)

Consider the game with payoff matrix

|     | $l$     | $r$     |
|-----|---------|---------|
| $U$ | $2, -2$ | $-1, 1$ |
| $D$ | $1, -1$ | $2, -2$ |

Given  $\sigma_2 = (q, 1 - q)$ , the LP conditions are

$$\begin{cases} 2q - 1(1 - q) \leq u_1^* & \implies 3q - 1 \leq u_1^* \\ 1q + 2(1 - q) \leq u_1^* & \implies 2 - q \leq u_1^* \end{cases} \quad (10)$$

Then  $\min_q u_1^*(q)$  subject to such conditions is computed imposing marginality, i.e.  $3q - 1 = 2 - q$ , which gives  $q = \frac{3}{4}$ .

# Computing NE of two-player general-sum games

For general-sum normal-form two-player games, the NE conditions can be written as **Quadratic Program**:

$$\begin{aligned} & \underset{\sigma_1, \sigma_2, u_1^*, u_2^*}{\text{maximize}} \quad \sum_{a_1, a_2} \sigma_1(a_1) [u_1(a_1, a_2) + u_2(a_1, a_2)] \sigma_2(a_2) - u_1^* - u_2^* \\ & \text{subject to} \quad \begin{cases} \sum_{a_2 \in A_2} u_1(a_1, a_2) \sigma_2(a_2) \leq u_1^*, & \forall a_1 \in A_1 \\ \sum_{a_1 \in A_1} u_2(a_1, a_2) \sigma_1(a_1) \leq u_2^*, & \forall a_2 \in A_2 \\ \sigma_1(a_1) \geq 0, & \forall a_1 \in A_1 \\ \sigma_2(a_2) \geq 0, & \forall a_2 \in A_2 \\ \sum_{a_1 \in A_1} \sigma_1(a_1) = 1 \\ \sum_{a_2 \in A_2} \sigma_2(a_2) = 1 \end{cases} \end{aligned} \quad (11)$$

which is however non-convex in general. Values  $u_1^*$  and  $u_2^*$  are best-response values in a NE.

# Computing NE of two-player general-sum games

Rescaling variables by  $u_1^*$  (resp.  $u_2^*$ ) and introducing slack variables, the QP is equivalent to the following **Linear Complementarity Problem**:

$$\begin{cases} \sum_{a_2 \in A_2} u_1(a_1, a_2) \sigma_2(a_2) + r(a_1) = 1, & \forall a_1 \in A_1 \\ \sum_{a_1 \in A_1} u_2(a_1, a_2) \sigma_1(a_1) + s(a_2) = 1, & \forall a_2 \in A_2 \\ \sigma_1(a_1) \geq 0, r(a_1) \geq 0, & \forall a_1 \in A_1 \\ \sigma_2(a_2) \geq 0, s(a_2) \geq 0, & \forall a_2 \in A_2 \\ \sum_{a_1 \in A_1} \sigma_1(a_1) r(a_1) = 0 \\ \sum_{a_2 \in A_2} \sigma_2(a_2) s(a_2) = 0 \end{cases}$$

that means  $w = Mz + q$  with  $z \geq 0$ ,  $w \geq 0$  and  $z^\top w = 0$ , with

$$w = \begin{pmatrix} r \\ s \end{pmatrix}, \quad M = \begin{pmatrix} 0 & -U_1 \\ -U_2^\top & 0 \end{pmatrix} \quad z = \begin{pmatrix} \sigma_1 \\ \sigma_2 \end{pmatrix}, \quad q = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

and the matrix  $U_i = \{u_i(a_i, a_{-i})\} \in \mathbb{R}^{|A_1| \times |A_2|}$ .

# Computing NE in N-player general-sum games

In multi-player games (any  $N > 2$  game is equivalent to a 3-player game), LCPs become *non-linear* and few algorithms can be generalized.

Alternatively, define an objective function

$$f(\sigma) = \sum_{i \in \mathcal{N}} \sum_{a'_i \in A_i} (\max(0, u_i(a'_i, \sigma_{-i}) - u_i(\sigma)))^2, \quad (12)$$

NE  $\sigma^*$  are solutions of the ***non-linear constrained optimization problem***

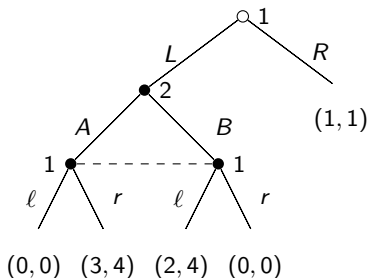
$$\begin{aligned} & \underset{\sigma}{\text{minimize}} && f(\sigma) \\ & \text{subject to} && \begin{cases} \sum_{a_i \in A_i} \sigma_i(a_i) = 1 & \forall i \in \mathcal{N} \\ \sigma_i(a_i) \geq 0 & \forall i \in \mathcal{N}, \forall a_i \in A_i. \end{cases} \end{aligned} \quad (13)$$

Only *global minima* of  $f(\sigma)$  are N.E., not the local minima.

# Computing NE in extensive-form games

While SPE can be efficiently computed using backward induction (dynamic programming), computing Nash equilibria in extensive-form games is apparently demanding (e.g. passing to normal form).

## Sequence-form representation



|             | $\emptyset$ | A    | B    |
|-------------|-------------|------|------|
| $\emptyset$ | 0, 0        | 0, 0 | 0, 0 |
| L           | 0, 0        | 0, 0 | 0, 0 |
| R           | 1, 1        | 0, 0 | 0, 0 |
| L $\ell$    | 0, 0        | 0, 0 | 2, 4 |
| Lr          | 0, 0        | 2, 4 | 0, 0 |

Sequences:  $s_1 \in S_1 = \{\emptyset, L, R, L\ell, Lr\}$  and  $s_2 \in S_2 = \{\emptyset, A, B\}$ .

# Computing NE in extensive-form games

We can define a **realization plan**  $r_i$  for every behavioral strategy  $\beta_i$  of player  $i$ , just associating to every sequence available to  $i$  the corresponding probability according to  $\beta_i$ .

In the previous game,  $\beta_1 = ((p_1, 1 - p_1), (p_2, 1 - p_2))$  and  $\beta_2 = (q, 1 - q)$  are mapped into

$$\begin{cases} r_1(\emptyset) = 1 \\ r_1(L) = p_1, & r_1(R) = 1 - p_1 \\ r_1(L\ell) = p_1 p_2, & r_1(Lr) = p_1(1 - p_2) \\ r_2(\emptyset) = 1 \\ r_2(A) = q, & r_2(B) = 1 - q \end{cases}$$

The problems of computing NE in two-player extensive-form games can be formulated as linear program (zero-sum case) or linear complementarity problems (general-sum case) on realization plans.

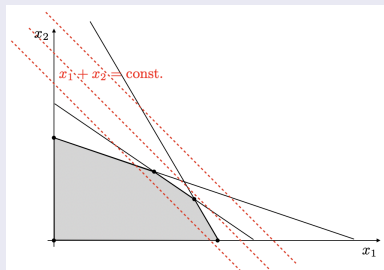


# Geometry of a Linear Program

## Example

Consider the LP,

$$\begin{array}{ll}\text{maximize}_{x_1, x_2} & x_1 + x_2 \\ \text{subject to} & \begin{cases} 2x_1 + 3x_2 \leq 12 \\ x_1 + 3x_2 \leq 9 \\ 2x_1 + x_2 \leq 10 \\ x_i \geq 0, \quad i = 1, 2 \end{cases}\end{array}$$



The optimal value corresponds to the vertex  $(9/2, 1)$ , solution of

$$\begin{cases} 2x_1 + 3x_2 = 12 \\ 2x_1 + x_2 = 10 \end{cases}$$

# Geometry of a Linear Program

## Remark

A LP has no solution if there is *no feasible* region.

For instance:

$$\begin{array}{ll}\text{minimize} & x_1 + x_2 \\ & x_1, x_2 \\ \text{subject to} & \begin{cases} x_1 + 2x_2 \leq 8 \\ 3x_1 + 2x_2 \leq 12 \\ x_1 + 3x_2 \geq 13 \\ x_i \geq 0, \quad i = 1, 2 \end{cases}\end{array}$$

there is no intersection of the half planes.

## Remark

The feasible region can be *unbounded*.

For instance:

$$\begin{array}{ll}\text{minimize} & 2x_1 - x_2 \\ & x_1, x_2 \\ \text{subject to} & \begin{cases} x_1 - x_2 \leq 1 \\ 2x_1 + x_2 \geq 6 \\ x_i \geq 0, \quad i = 1, 2 \end{cases}\end{array}$$

there is no finite solution to the LP.

## Remark

The LP admit an *infinite* number of finite solutions (degeneracy).

For instance:

$$\begin{array}{ll} \underset{x_1, x_2}{\text{minimize}} & 2x_1 + 2x_2 \\ \text{subject to} & \begin{cases} x_1 \leq 200 \\ 2x_2 \leq 300 \\ x_1 + x_2 \leq 400 \\ x_i \geq 0, \quad i = 1, 2 \end{cases} \end{array}$$

All points of an edge of the feasible region are solutions ( $x_1 + x_2 = 400$ ).

# Geometry of a Linear Program

A **polyhedron**  $P \subseteq \mathbb{R}^n$  is any set of the form  $P = \{x \in \mathbb{R}^n | Ax \leq b\}$  where  $A \in \mathbb{R}^{m \times n}$  and  $b \in \mathbb{R}^m$ . Since each row  $a_j^\top x \leq b_j$  represents half-space, a polyhedron is the intersection of a finite number of half-spaces.

If the polyhedron is bounded it is called a **polytope**. A polytope has a finite number of vertices. A bounded feasible region of a LP is a polytope.

## Proposition

*If there exists an optimal solution of a linear program then there exists one that is an extreme point (vertex of the polytope).*

## Remark

The number of vertices is finite but possibly *exponential* in the number of variables.

# Solving a LP using the Simplex Algorithm

The **Simplex Algorithm** (G. Dantzig, 1947) is a local search algorithm that starts at some vertex, pivots to a neighboring one, repeating the process until an optimum is found.

SA is not polynomial, but very efficient in most practical cases (*interior-point methods* are provably worst-case polynomial but worse on average).

IDEA: Suppose the origin vertex  $x = 0$  is feasible, then it is optimal for  $\max_x c^\top x$  only if  $c_i \leq 0, \forall i$ .

If it exists some  $c_i > 0$ , it is convenient to release the constraint  $x_i = 0$  increasing  $x_i$  until some other inequality becomes tight (reaching another vertex). The new vertex is moved in the origin with a change of basis.

The process is repeated, until no further improvement can be obtained (i.e.  $c_i \leq 0, \forall i$  in the current basis).

# Solving a LP using the Simplex Algorithm

## Example

Consider the LP

$$\begin{array}{ll}\text{maximize} & 2x_1 + 5x_2 \\ & x_1, x_2 \\ \text{subject to} & \begin{cases} 2x_1 - x_2 \leq 4 & (1) \\ x_1 + 2x_2 \leq 9 & (2) \\ -x_1 + x_2 \leq 3 & (3) \\ x_1 \geq 0 & (4) \\ x_2 \geq 0 & (5) \end{cases}\end{array}$$

The origin  $x_1 = 0, x_2 = 0$  corresponds to vertex (4), (5).

# Solving a LP using the Simplex Algorithm

## Example (cont'd)

Release the tight constraint (5) (arbitrary choice) increasing variable  $x_2$ .  
Check which other inequality first becomes tight among

$$\begin{cases} x_2 \geq 2x_1 - 4 & \implies & x_2 \geq -4 \\ x_2 \leq \frac{9}{2} - \frac{x_1}{2} & \implies & x_2 \leq \frac{9}{2} \\ x_2 \leq 3 + x_1 & \implies & x_2 \leq 3 \end{cases} \quad (*)$$

The variable  $x_2$  is thus increased up to  $x_2 = 3$ .

The new vertex reached is specified by constraints (3) and (4).



# Solving a LP using the Simplex Algorithm

## Example (cont'd)

The current vertex is moved to the origin of basis  $\{y_1, y_2\}$ ,

$$\begin{cases} y_1 = x_1 \\ y_2 = 3 + x_1 - x_2 \end{cases}$$

and the LP becomes

$$\underset{y_1, y_2}{\text{maximize}} \quad 15 + 7y_1 - 5y_2$$

$$\text{subject to } \begin{cases} y_1 + y_2 \leq 7 & (1) \\ 3y_1 - 2x_2 \leq 3 & (2) \\ y_2 \geq 0 & (3) \\ y_1 \geq 0 & (4) \\ -y_1 + y_2 \leq 3 & (5) \end{cases}$$

# Solving a LP using the Simplex Algorithm

## Example (cont'd)

Release the tight constraint (4) increasing variable  $y_1$ .  
Check which other inequality first becomes tight among

$$\begin{cases} y_1 \leq 7 - y_2 \\ y_1 \leq 1 + \frac{2}{3}y_2 \end{cases} \quad (*)$$

The variable  $y_1$  is thus increased up to  $y_1 = 1$ .

The new vertex reached is specified by constraints (2) and (3).

# Solving a LP using the Simplex Algorithm

## Example (cont'd)

The current vertex is moved to the origin of basis  $\{z_1, z_2\}$

$$\begin{cases} z_1 = 3 - 3y_1 + 2y_2 \\ z_2 = y_2 \end{cases}$$

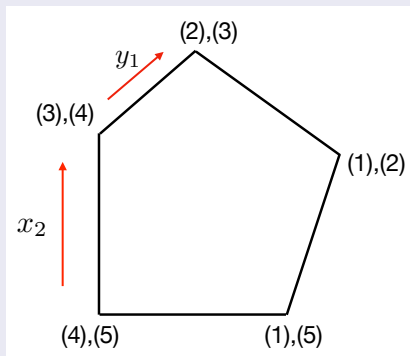
and the LP becomes

$$\begin{aligned} & \underset{z_1, z_2}{\text{maximize}} \quad 22 - \frac{7}{3}z_1 - \frac{1}{3}z_2 \\ & \text{subject to} \quad \begin{cases} -\frac{1}{3}z_1 + \frac{5}{3}z_2 \leq 6 & (1) \\ z_1 \geq 0 & (2) \\ z_2 \geq 0 & (3) \\ \frac{1}{3}z_1 - \frac{2}{3}z_2 \leq 1 & (4) \\ \frac{1}{3}z_1 + \frac{1}{3}z_2 \leq 4 & (5) \end{cases} \end{aligned}$$

# Solving a LP using the Simplex Algorithm

## Example (cont'd)

The origin  $z_1 = z_2 = 0$  is the optimal solution because  $c_i < 0, \forall i$ .  
In the original problem, the optimal solution is  $(x_1, x_2) = (1, 4)$  corresponding to vertex (2), (3).



# Solving a LP using the Simplex Algorithm

## Final remarks:

- *Initial vertex*: If  $\exists j$  s.t.  $b_j < 0$ , the origin is not feasible, but the problem of finding the initial vertex can be reduced to a LP for which the origin is feasible.
- *Degeneracy*: Small perturbations can be introduced to prevent the algorithm from hopping forever between vertices corresponding to the same value.
- *Unboundedness*: If releasing a constraint, there is no upper bound for the increasing variable, it means that the space of solutions contains a line across which the objective function can diverge.
- *Running time*:  $O(nm)$  time complexity per iteration, but the number of vertices grows exponentially with  $n$  and  $m$ .

# Geometry of a Linear Complementarity Problem

## Definition

The *best-response polytope* of player  $i$ 's is a set  $\bar{P}_i$  of mixed strategies together with the upper envelope of  $-i$ 's expected payoff, defined as

$$\bar{P}_i = \left\{ (\sigma_i, v_{-i}) \mid \sigma_i(a_i) \geq 0, \forall a_i \in A_i, \sum_{a_i \in A_i} \sigma_i(a_i) = 1, \right. \\ \left. \sum_{a_i \in A_i} u_{-i}(a_i, a_{-i}) \sigma_i(a_i) \leq v_{-i}, \forall a_{-i} \in A_{-i} \right\} \quad (14)$$

After rescaling strategies  $\sigma_1/v_2 \rightarrow \sigma_1$  and  $\sigma_2/v_1 \rightarrow \sigma_2$ ,

$$P_i = \left\{ \sigma_i \in \mathbb{R}^{|A_i|} \mid \sigma_i(a_i) \geq 0, \forall a_i \in A_i, \sum_{a_i \in A_i} \sigma_i(a_i) u_{-i}(a_i, a_{-i}) \leq 1, \forall a_{-i} \in A_{-i} \right\}. \quad (15)$$

A vertex of polytope  $P_i$  is the intersection of  $|A_i|$  saturated inequalities, it can be labeled by them.

The total number of labels is equal to  $|A_1| + |A_2|$  for both agents.

# Geometry of a Linear Complementarity Problem

## Example

Consider again

|     | $L$  | $R$  |
|-----|------|------|
| $T$ | 3, 3 | 3, 2 |
| $M$ | 2, 2 | 5, 6 |
| $D$ | 0, 3 | 6, 1 |

the best-response polytopes  $P_2$  and  $P_1$  are defined by

$$P_2 : \quad 3y_1 + 3y_2 \leq 1 \quad (1)$$

$$2y_1 + 5y_2 \leq 1 \quad (2)$$

$$6y_2 \leq 1 \quad (3)$$

$$y_1 \geq 0 \quad (4)$$

$$y_2 \geq 0 \quad (5)$$

$$P_1 : \quad 3x_1 + 2x_2 + 3x_3 \leq 1 \quad (4)$$

$$2x_1 + 6x_2 + x_3 \leq 1 \quad (5)$$

$$x_1 \geq 0 \quad (1)$$

$$x_2 \geq 0 \quad (2)$$

$$x_3 \geq 0 \quad (3)$$

# Geometry of a Linear Complementarity Problem

## Example (cont'd)

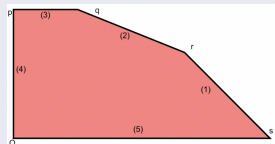
Vertices of  $P_2$  in addition to  $O = (0,0)$ :

$p = (0, 1/6)$ , labeled by conditions (3) and (4),

$q = (1/12, 1/6)$ , labeled by (3) and (2),

$r = (2/9, 1/9)$ , labeled by (2) and (1),

$s = (1/3, 0)$ , labeled by (1) and (5).



Vertices of  $P_1$  in addition to  $O = (0,0,0)$ :

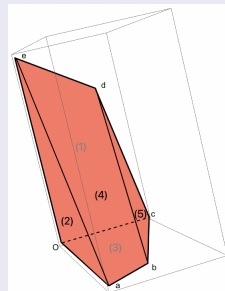
$a = (1/3, 0, 0)$ , labeled by (2), (3), (4),

$b = (2/7, 1/14, 0)$ , labeled by (3), (4), (5),

$c = (0, 1/6, 0)$ , labeled by (1), (3), (5),

$d = (0, 1/8, 1/4)$ , labeled by (1), (4), (5),

$e = (0, 0, 1/3)$ , labeled by (1), (2), (4).





# Geometry of a Linear Complementarity Problem

## Definition

A vertex pair  $(\sigma_1, \sigma_2)$  of mixed strategies is *completely labeled* if every label  $k \in A_1 \cup A_2$  appears as a label of either  $\sigma_1$  or  $\sigma_2$ .

## Proposition

A mixed strategy profile  $(\sigma_1, \sigma_2)$  is a Nash equilibrium if and only if the corresponding vertex pair  $(\sigma_1, \sigma_2) \in P_1 \times P_2$  is completely labeled.

We could compute the best-response polytopes, then for every pair of vertices in the polytopes, verify if they are N.E. by checking complete labeling.

However, the number of vertices can be exponential in the number of actions.

# Solving a LCP using Lemke-Howson Algorithm

*Geometric idea:* build a path from the origin pair  $(O, O)$  moving along edges in  $P_1$  and  $P_2$  alternately until a completely labeled vertex is found.

## ***Lemke-Howson Algorithm:***

- start from the completely labeled vertex pair at the origins of the best-response polytopes;
- an arbitrary label is chosen on one of the two vertices;
- repeat until a non-trivial completely labeled vertex pair is found:
  - the label is dropped via a *pivoting operation*, i.e. by moving to a vertex adjacent to the current one that does not contain the hyperplane associated with that label;
  - a duplicated label is identified, on the other polytope where it appears;
- normalizing the values of the completely labeled vertex a Nash equilibrium is obtained.

# Solving a LCP using Lemke-Howson Algorithm

*Algebraic formulation:* introduce non-negative slack variables  $r \in \mathbb{R}^{|A_1|}$  and  $s \in \mathbb{R}^{|A_2|}$ , the polytopes are defined by a system of equations with non-negativity constraints,

$$\left\{ \begin{array}{ll} \sum_{a_2 \in A_2} u_1(a_1, a_2) \sigma_2(a_2) + r(a_1) = 1, & \forall a_1 \in A_1 \\ \sum_{a_1 \in A_1} u_2(a_1, a_2) \sigma_1(a_1) + s(a_2) = 1, & \forall a_2 \in A_2 \\ \sigma_1(a_1) \geq 0, r(a_1) \geq 0, & \forall a_1 \in A_1 \\ \sigma_2(a_2) \geq 0, s(a_2) \geq 0, & \forall a_2 \in A_2 \\ \sum_{a_1 \in A_1} \sigma_1(a_1) r(a_1) = 0 \\ \sum_{a_2 \in A_2} \sigma_2(a_2) s(a_2) = 0 \end{array} \right.$$

which is a *Linear Complementarity Problem*.

The solution, via Integer Pivoting (Simplex Algorithm), realizes the same alternated path described in the geometric representation.

# Solving a LCP using Lemke-Howson Algorithm

## Example

Consider again

|     | $L$  | $R$  |
|-----|------|------|
| $T$ | 3, 3 | 3, 2 |
| $M$ | 2, 2 | 5, 6 |
| $D$ | 0, 3 | 6, 1 |

the LCP representation is

$$\left\{ \begin{array}{l} 3y_1 + 3y_2 + r_1 = 1 \\ 2y_1 + 5y_2 + r_2 = 1 \\ 6y_2 + r_3 = 1 \\ 3x_1 + 2x_2 + 3x_3 + s_1 = 1 \\ 2x_1 + 6x_2 + x_3 + s_2 = 1. \end{array} \right. \quad (16)$$

# Solving a LCP using Lemke-Howson Algorithm

## Example (cont'd)

- Start from the *basis*  $\{r_1, r_2, r_3, s_1, s_2\}$  such that

$$\begin{cases} r_1 = 1 - 3y_1 - 3y_2 \\ r_2 = 1 - 2y_1 - 5y_2 \\ r_3 = 1 - 6y_2 \\ s_1 = 1 - 3x_1 - 2x_2 - 3x_3 \\ s_2 = 1 - 2x_1 - 6x_2 - x_3, \end{cases} \quad (17)$$

- Apply integer pivoting:
  - Choose a new variable to enter the basis, e.g.  $x_2$  replacing  $s_2$
  - The dual variable of  $s_2$  is  $y_2$ ; it must enter replacing  $r_3$
  - The dual variable of  $r_3$  is  $x_3$ ; it must enter replacing  $s_1$ ;
  - The dual variable of  $s_1$  is  $y_1$ ; it must enter replacing  $r_2$ .

# Solving a LCP using Lemke-Howson Algorithm

## Example (cont'd)

- Once a *complete basis* is obtained, e.g.  $\{r_1, x_2, x_3, y_1, y_2\}$

$$\begin{cases} r_1 = \frac{1}{4} + \frac{3}{2}r_2 - \frac{3}{4}r_3 \\ x_2 = \frac{3}{24} - \frac{3}{16}x_1 + \frac{1}{16}s_1 - \frac{7}{48}s_2 \\ x_3 = \frac{1}{4} - \frac{7}{8}x_1 - \frac{3}{8}s_1 + \frac{1}{8}s_2 \\ y_1 = \frac{1}{12} - \frac{1}{2}r_2 + \frac{5}{12}r_3 \\ y_2 = \frac{1}{6} - \frac{1}{6}r_3 \end{cases} \quad (18)$$

set to zero slack variables which are not in the basis, which gives

$$\implies r_1 = \frac{1}{4}, \quad y_1 = \frac{1}{12}, \quad y_2 = \frac{1}{6}, \quad x_2 = \frac{1}{8}, \quad x_3 = \frac{1}{4}.$$

- Normalize to obtain a NE with  $\sigma_1 = \left(0, \frac{1}{3}, \frac{2}{3}\right)$  and  $\sigma_2 = \left(\frac{1}{3}, \frac{2}{3}\right)$ .

# Computational complexity

How much difficult is to find a N.E.? The Lemke-Howson algorithm generates paths on the vertices of a polytope.

## Proposition

*Finding a sample Nash equilibrium of a general-sum finite game with two or more players is PPAD-complete.*

## Remark (PPAD – *Parity Preserving Arguments on Directed Graphs*)

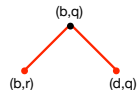
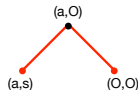
It is a complexity class associated with finding a sink node in a directed graph with an exponential number of nodes, with in-degree and out-degree at most 1, and a given source node.

Finding N.E. with additional constraints is NP-hard as well as finding pure-strategy N.E.

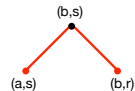
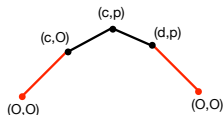
# Computational complexity: PPAD example

Lemke-Howson graph (of previous example): consider  $x \in X$ ,  $y \in Y$  such that  $(x, y)$  is almost completely labeled, then add completely labeled vertices.

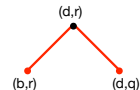
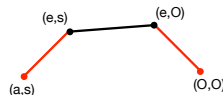
label 1:  $a = (2, 3, 4) - O = (4, 5)$   
 $b = (3, 4, 5) - q = (3, 2)$



label 2:  $c = (3, 1, 5) - O = (4, 5)$   
 $c = (3, 4, 5) - p = (3, 4)$   
 $d = (1, 4, 5) - p = (3, 4)$   
 $b = (3, 4, 5) - s = (1, 5)$



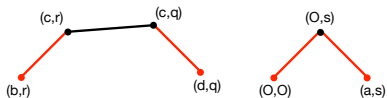
label 3:  $e = (1, 2, 4) - O = (4, 5)$   
 $d = (1, 4, 5) - r = (2, 1)$   
 $e = (1, 2, 4) - s = (1, 5)$



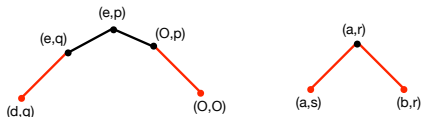


# Computational complexity: PPA example

label 4:  $c = (3,1,5) - q = (2,3)$   
 $c = (3,1,5) - r = (1,2)$   
 $O = (1,2,3) - s = (1,5)$



label 5:  $O = (1,2,3) - p = (3,4)$   
 $e = (1,2,4) - p = (3,4)$   
 $e = (1,2,4) - q = (2,3)$   
 $a = (2,3,4) - r = (1,2)$



Only nodes of degree 1 and 2 appear. Completely labeled nodes have degree 1. There is an even number of nodes of degree 1, but one is the artificial node  $(O,O)$ .

Hence, **the number of N.E. is odd.**

Giving a direction to paths between completely labeled nodes (relation with PPA class).